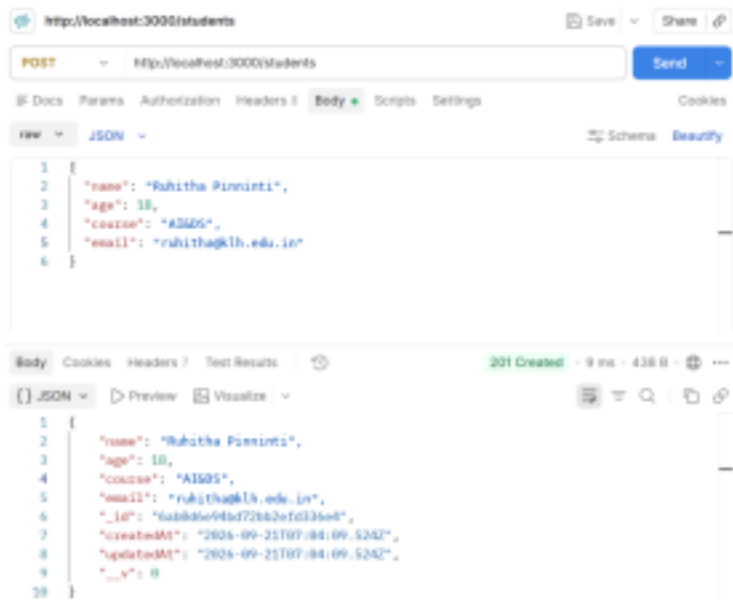


WEEK-7 SKILL — Student Records

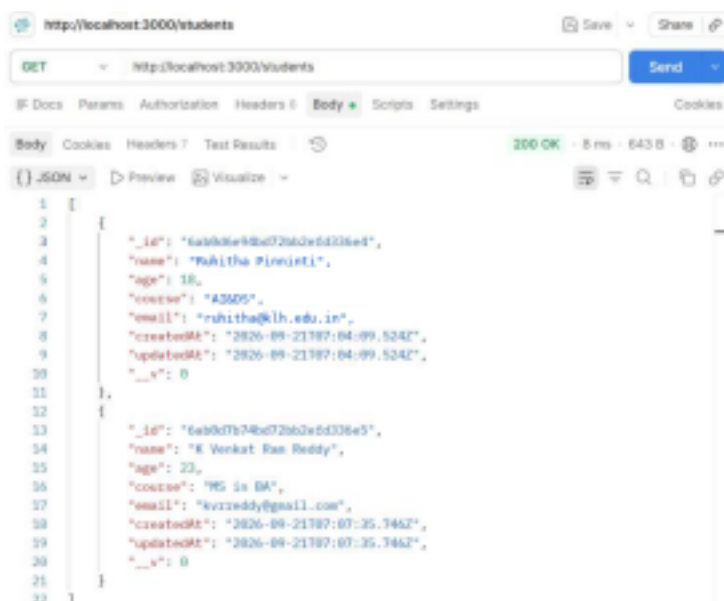
CRUD API POST:



After POST in mongosh:

```
student_records> db.students.find()
[
  {
    _id: ObjectId('6ab8d6e94bd72bb2efd336e4'),
    name: 'Ruhitha Pinninti',
    age: 18,
    course: 'AISDS',
    email: 'ruhitha@klh.edu.in',
    createdAt: ISODate('2026-09-21T07:04:09.524Z'),
    updatedAt: ISODate('2026-09-21T07:04:09.524Z'),
    __v: 0
  }
]
```

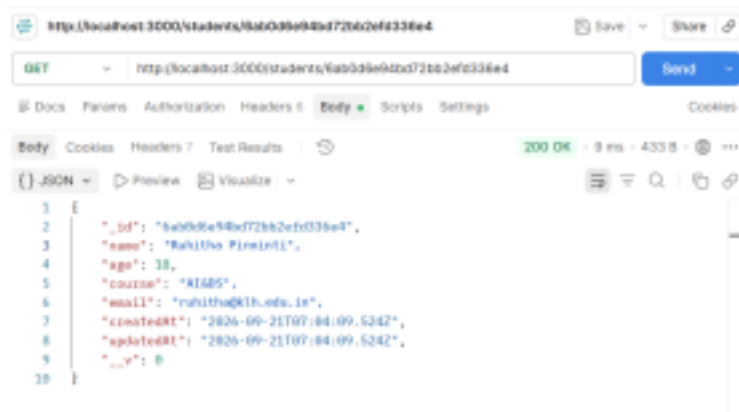
GET all data:



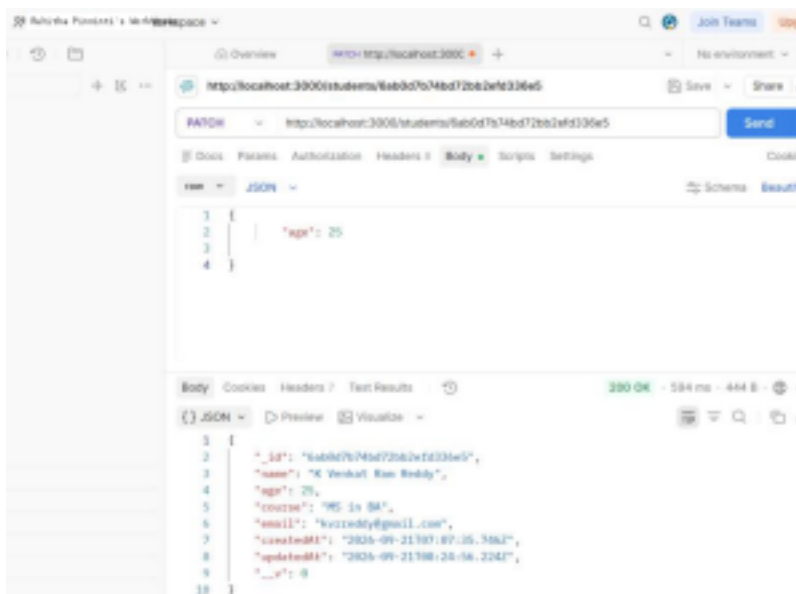
GET all data in mongosh:

```
student_records> db.students.find()
[
  {
    _id: ObjectId('6ab0d6e94bd72bb2efd336e4'),
    name: 'Ruhitha Pinninti',
    age: 18,
    course: 'AI&DS',
    email: 'ruhitha@klh.edu.in',
    createdAt: ISODate('2026-09-21T07:04:09.524Z'),
    updatedAt: ISODate('2026-09-21T07:04:09.524Z'),
    __v: 0
  },
  {
    _id: ObjectId('6ab0d7b74bd72bb2efd336e5'),
    name: 'K Venkat Ram Reddy',
    age: 23,
    course: 'MS in BA',
    email: 'kvrreddy@gmail.com',
    createdAt: ISODate('2026-09-21T07:07:35.746Z'),
    updatedAt: ISODate('2026-09-21T07:07:35.746Z'),
    __v: 0
  }
]
```

GET one with specific id:



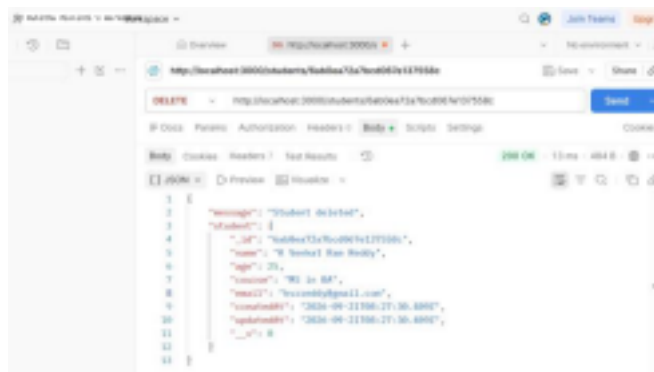
PATCH changed age from 23 to 25:



After modification of age output in mongosh:

```
{
  _id: ObjectId('6ab0ea72a7bcd067e137558c'),
  name: 'K Venkat Ram Reddy',
  age: 25,
  course: 'MS in BA',
  email: 'kvrreddy@gmail.com',
  createdAt: ISODate('2026-09-21T08:27:30.409Z'),
  updatedAt: ISODate('2026-09-21T08:27:30.409Z'),
  __v: 0
}
```

DELETE:



Observation of output in mongosh:

```
student_records> db.students.find()
[
  {
    _id: ObjectId('6ab0d6e94bd72bb2efd336e4'),
    name: 'Ruhitha Pinninti',
    age: 17,
    course: 'CSIT',
    email: 'ruhitha@klh.edu.in',
    createdAt: ISODate('2026-09-21T07:04:09.524Z'),
    updatedAt: ISODate('2026-09-21T07:04:09.524Z'),
    __v: 0
  }
]
student_records>
```

Folder Structure:

Week_7_StudentRecordsAPI

```
├── controllers
│   └── studentController.js
├── models
│   └── Student.js
├── routes
│   └── studentRoutes.js
├── server.js
└── package.json
```

Code:

package.json

```
{
```

```

"name":
"week_7_studentrecordsapi",
"version": "1.0.0",
"type": "module",
"description": "",
"main": "server.js",
"scripts": {
  "start": "node server.js",
  "dev": "nodemon server.js"
},
"keywords": [],
"author": "",
"license": "ISC",
"dependencies": {
  "express": "^5.2.1",
  "mongoose": "^9.10.1"
},
"devDependencies": {
  "nodemon": "^3.1.14"
}
}

```

models/Student.js

```

import mongoose from 'mongoose';

const studentSchema = new
mongoose.Schema({ name: {
  type: String,
  required: true,
  trim: true
},
age: {
  type: Number,
  required: true,
  min: 16,
  max: 100
},
course: {
  type: String,
  required: true,
  trim: true
},
email: {
  type: String,
  required: true,
  unique: true,
  lowercase: true

```

```

    }
  }, {
    timestamps: true
  });

const Student = mongoose.model('Student',
studentSchema); export default Student;

```

controllers/studentController.js

```

import Student from '../models/Student.js';

export const createStudent = async (req, res) => {
  try {
    const student = await Student.create(req.body);
    res.status(201).json(student);
  } catch (error) {
    res.status(400).json({ error: error.message });
  }
};

export const getAllStudents = async (req, res) => {
  try {
    const students = await Student.find();
    res.status(200).json(students);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

export const getStudentById = async (req, res) => {
  try {
    const student = await Student.findById(req.params.id);

    if (!student)
      return res.status(404).json({ message: 'Student not found' });

    res.status(200).json(student);
  } catch (error) {
    res.status(400).json({ error: error.message });
  }
};

export const updateStudent = async (req, res) => {
  try {
    const student = await Student.findByIdAndUpdate(
      req.params.id,

```

```

    req.body,
    { new: true, runValidators: true }
  );

  if (!student)
    return res.status(404).json({ message: 'Student not found' });

  res.status(200).json(student);
} catch (error) {
  res.status(400).json({ error: error.message });
}
};

export const deleteStudent = async (req, res) => {
  try {
    const student = await Student.findByIdAndDelete(req.params.id);

    if (!student)
      return res.status(404).json({ message: 'Student not found' });

    res.status(200).json({
      message: 'Student deleted',
      student
    });
  } catch (error) {
    res.status(400).json({ error: error.message });
  }
};

```

routes/studentRoutes.js

```

import express from 'express';

import {
  createStudent,
  getAllStudents,
  getStudentById,
  updateStudent,
  deleteStudent
} from '../controllers/studentController.js';

const router = express.Router();

router.post('/', createStudent);
router.get('/', getAllStudents);
router.get('/:id', getStudentById);
router.patch('/:id', updateStudent);
router.delete('/:id', deleteStudent);

```

```
export default router;
```

server.js

```
import express from 'express';
import mongoose from 'mongoose';
import studentRoutes from './routes/studentRoutes.js';

const app = express();
const PORT = 3000;

app.use(express.json());

app.get('/', (req, res) => {
  res.json({
    message: 'Student Records API is running'
  });
});

app.use('/students', studentRoutes);

mongoose.connect('mongodb://127.0.0.1:27017/student_records'
) .then(() => {
  console.log('MongoDB connected');

  app.listen(PORT, () => {
    console.log(`Server running on
    http://localhost:${PORT}`); });
})
.catch((error) => {
  console.error('MongoDB connection failed:',
error.message); });
```